

# ESE 224 Final Project Report

## Stock Market Portfolio Simulator

Irfan Chowdhury  
Biheng Chen

ESE 224

This report documents the design, implementation, testing, and results of a C++ stock market simulation program built using data structures and object-oriented programming concepts from ESE 224.

# Introduction

The Stock Market Portfolio Simulator is a C++ command-line program designed to load historical market data, manage a simple investment portfolio, organize stock performance through data structures, and test different long-term investing strategies. The program uses real historical CSV data for SPX, AMZN, and NVDA and evaluates strategy behavior from 2000 to the beginning of 2020. Since the data filter ends at 2020-01-01, the strategy simulations effectively run through the end of 2019.

The project was built around the main topics covered in ESE 224: classes and objects, linked lists, iterators, queues, stacks, Binary Search Trees, file handling, and modular program design.

```
=====
ESE 224 Stocksim - Student Login
=====
Enter your full name: Irfan Chowdhury
Enter your student ID: 115901773

welcome, Irfan Chowdhury!

===== Stocksim: Historical Market Analyzer =====
Student: Irfan Chowdhury | ID: 115901773
-----
[1] Load stock data from csv
[2] Display price history (linked list traversal)
[3] Search by date range
[4] Find stocks by return range (BST range query)
[5] Insert stock into performance BST
[6] Display BST (inorder / preorder / postorder)
[7] Add stock to portfolio
[8] Remove stock from portfolio
[9] Queue a pending order
[10] Execute next pending order
[11] Undo last trade
[12] Run strategy simulation
[13] Compare all strategies head-to-head
[14] Display portfolio summary
[15] Display full trade history
[0] Exit
-----
Enter choice: 1
```

Figure 1. Main program menu and student login interface.

# Project Goals

- Load and parse historical stock market CSV files.
- Store price history using a linked list and traverse it using a custom iterator.
- Use a Binary Search Tree to organize stocks by return performance.
- Implement portfolio operations such as buy, sell, queue order, execute order, and undo trade.
- Implement and compare four investment strategies: Fixed SIP, Dynamic SIP, Golden Cross, and Momentum.
- Report useful performance metrics, including final value, total invested, total return, CAGR, maximum drawdown, and total trades.

# System Architecture

The project is organized into separate source and header files. This keeps each class responsible for one main part of the system and makes the program easier to maintain. The main program controls the menu, while the classes handle parsing, storage, strategy logic, portfolio operations, and performance analysis.

| Component                                   | Purpose                                                                           |
|---------------------------------------------|-----------------------------------------------------------------------------------|
| CSVParser                                   | Reads CSV files, parses columns, filters dates, and creates PriceHistory objects. |
| PriceHistory / PriceNode                    | Stores historical price data in linked-list nodes.                                |
| Stock and ETF                               | Represent market assets and load their historical data.                           |
| StockManager<T>                             | Template-based manager for Stock and ETF objects.                                 |
| Portfolio                                   | Tracks cash, holdings, portfolio value, and trade actions.                        |
| TradeStack                                  | Stores recent trades so the most recent action can be undone.                     |
| OrderQueue                                  | Stores pending orders using FIFO behavior.                                        |
| StockBST                                    | Organizes annual stock returns and supports traversal/range searching.            |
| TradingStrategy                             | Abstract base class for strategy simulations.                                     |
| FixedSIP, DynamicSIP, GoldenCross, Momentum | Derived strategy classes implementing separate investment rules.                  |

# CSV Data Loading and Parsing

The CSV parser reads market data from files in the data folder. It identifies the important columns from the header row, which allows it to handle CSV files with different column ordering. It then filters rows to the project window and appends valid rows into a PriceHistory linked list.

A major fix was needed for the volume column. SPX trading volume values can be larger than the normal 32-bit integer range on Windows. Because of this, the project uses long long and stoll() for volume parsing instead of long and stol(). This fixed skipped rows and allowed SPX to correctly load 5031 trading days.

```
Enter choice: 1
Choose data to load:
1. SPX
2. AMZN
3. NVDA
4. All
Enter choice: 4
SPX: loaded 5031 trading days.
AMZN: loaded 5031 trading days.
NVDA: loaded 5031 trading days.
Data loading complete.
```

Figure 2. Successful loading of SPX, AMZN, and NVDA data.

# Data Structures Used

## Linked List and Iterator

Historical prices are stored in a linked list. Each PriceNode contains date, open, high, low, close, volume, and links to neighboring nodes. The iterator class allows the strategy code to traverse PriceHistory cleanly using begin() and end().

```
Enter choice: 2
Enter ticker: SPX
Enter start date (YYYY-MM-DD): 2000-01-01
Enter end date (YYYY-MM-DD): 2000-01-10
2000-01-03  open: 1469.25  High: 1478.00  Low: 1438.36  close: 1455.22  volume: 931800000
2000-01-04  open: 1455.22  High: 1455.22  Low: 1397.43  close: 1399.42  volume: 1009000000
2000-01-05  open: 1399.42  High: 1413.27  Low: 1377.68  close: 1402.11  volume: 1085500000
2000-01-06  open: 1402.11  High: 1411.90  Low: 1392.10  close: 1403.45  volume: 1092300000
2000-01-07  open: 1403.45  High: 1441.47  Low: 1400.73  close: 1441.47  volume: 1225200000
2000-01-10  open: 1441.47  High: 1464.36  Low: 1441.47  close: 1457.60  volume: 1064800000
```

Figure 3. Historical SPX data displayed through linked-list traversal.

## Binary Search Tree

The BST stores stock returns by year. This allows the program to insert stock performance values and search by a return range. The BST also supports inorder, preorder, and postorder display.

```
Enter choice: 6
1. Inorder
2. Preorder
3. Postorder
Enter choice: 1
AMZN (20.06) [2019]
NVDA (72.74) [2019]
```

Figure 4. BST inorder traversal after inserting AMZN and NVDA performance.

```
Enter choice: 4
Enter minimum return: 0
Enter maximum return: 100
AMZN | Return: 20.06%
NVDA | Return: 72.74%
```

Figure 5. BST range query for stock returns between 0% and 100%.

## Stack

The stack is used for undo functionality. The most recent trade is pushed to the stack, and undo reverses the most recent action first, which matches LIFO behavior.

## Queue

The queue is used for pending orders. Orders are processed in the same order they were added, which matches FIFO behavior.

## Portfolio System

The portfolio system keeps track of the student user, cash balance, current holdings, average cost, market value, and unrealized returns. The user can add shares, remove shares, queue pending orders, execute orders, and undo trades.

```
Enter choice: 14
Portfolio for 1
Cash balance: $9000.00
Holdings:
AMZN | Shares: 10 | Avg Cost: $100.00 | Current: $100.00 | Market Value: $1000.00 | Return: 0.00%
Total market value: $1000.00
Total portfolio value: $10000.00
Total unrealized return: 0.00%
```

Figure 6. Portfolio summary after buying AMZN shares.

```
Enter choice: 11
Undid BUY trade.
```

Figure 7. Undo operation using the trade stack.

```
Enter choice: 14
Portfolio for Irfan Chowdhury
Cash balance: $10000.00
No holdings.
```

Figure 8. Portfolio summary after undoing the trade.

# Investment Strategies

## Fixed SIP Strategy

Fixed SIP invests the same amount of capital once per month. It is the simplest baseline strategy because it ignores short-term price movement and consistently buys throughout the entire date range.

- Monthly capital: \$500
- Period: 2000 to 2020
- SPX total trades: 240
- SPX total invested: \$120,000.00

```
Enter choice: 12
Enter ticker (SPX, AMZN, NVDA): SPX
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

Choose strategy:
1. Fixed SIP
2. Dynamic SIP
3. Golden Cross
4. Momentum
Enter choice: 1
Strategy: Fixed SIP
Final value: $276886.13
Total invested: $120000.00
Total return: 130.74%
CAGR: 4.27%
Max drawdown: 48.72%
Total trades: 240
```

Figure 9. Fixed SIP strategy output for SPX.

## Dynamic SIP Strategy

Dynamic SIP adjusts the monthly investment depending on market conditions. In this implementation, the strategy increases investment during dips and reduces investment during strong rallies.

- Dip threshold: 10%
- Rally threshold: 20%
- Multiplier during dips: 2x

```
Enter choice: 12
Enter ticker (SPX, AMZN, NVDA): SPX
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

Choose strategy:
1. Fixed SIP
2. Dynamic SIP
3. Golden Cross
4. Momentum
Enter choice: 2
Enter dip threshold (%): 10
Enter rally threshold (%): 20
Enter multiplier: 2
Strategy: Dynamic SIP (Dip 10%, Rally 20%, x2)
Final value: $325559.43
Total invested: $131250.00
Total return: 148.05%
CAGR: 4.65%
Max drawdown: 45.30%
Total trades: 240
```

Figure 10. Dynamic SIP strategy output for SPX.



## Golden Cross Strategy

The Golden Cross strategy compares a short-term moving average with a long-term moving average. The implementation uses 50-day and 200-day moving averages. When the 50-day average crosses above the 200-day average, the strategy enters the market. When it crosses below, the strategy exits or reduces exposure.

```
Enter choice: 12
Enter ticker (SPX, AMZN, NVDA): SPX
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

choose strategy:
1. Fixed SIP
2. Dynamic SIP
3. Golden Cross
4. Momentum
Enter choice: 3
Strategy: Golden Cross (50/200 MA)
Final value: $251898.38
Total invested: $120000.00
Total return: 109.92%
CAGR: 3.78%
Max drawdown: 16.05%
Total trades: 17
```

Figure 11. Golden Cross strategy output for SPX.

## Momentum Strategy

The Momentum strategy checks whether recent price movement is above a threshold. In the tests, a 5% momentum threshold was used. The goal is to follow strong upward trends while avoiding weaker periods.

```
Enter choice: 12
Enter ticker (SPX, AMZN, NVDA): SPX
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

Choose strategy:
1. Fixed SIP
2. Dynamic SIP
3. Golden Cross
4. Momentum
Enter choice: 4
Enter momentum threshold (%): 5
Strategy: 6-Month Momentum (5%)
Final value: $233042.46
Total invested: $120000.00
Total return: 94.20%
CAGR: 3.37%
Max drawdown: 19.21%
Total trades: 120
```

Figure 12. Momentum strategy output for SPX.

# Strategy Results and Comparison

The same monthly capital and date range were used for each strategy. The main results are summarized below. Table values are rounded for readability, while the screenshots show the exact terminal output. In general, Dynamic SIP produced the strongest return for the assets tested, while Golden Cross and Momentum reduced maximum drawdown for SPX compared to Fixed SIP.

## SPX Strategy Results

| Strategy     | Final Value | Total Invested | Total Return | CAGR  | Max Drawdown | Trades |
|--------------|-------------|----------------|--------------|-------|--------------|--------|
| Fixed SIP    | \$276.9K    | \$120.0K       | 130.74%      | 4.27% | 48.72%       | 240    |
| Dynamic SIP  | \$325.6K    | \$131.3K       | 148.05%      | 4.65% | 45.30%       | 240    |
| Golden Cross | \$251.9K    | \$120.0K       | 109.92%      | 3.78% | 16.05%       | 17     |
| Momentum     | \$233.0K    | \$120.0K       | 94.20%       | 3.37% | 19.21%       | 120    |

```
Enter choice: 13
Enter ticker (SPX, AMZN, NVDA): SPX
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

===== Strategy Comparison for SPX =====
Strategy: Fixed SIP
Final value: $276886.13
Total invested: $120000.00
Total return: 130.74%
CAGR: 4.27%
Max drawdown: 48.72%
Total trades: 240

Strategy: Dynamic SIP (Dip 10%, Rally 20%, x2)
Final value: $325559.43
Total invested: $131250.00
Total return: 148.05%
CAGR: 4.65%
Max drawdown: 45.30%
Total trades: 240

Strategy: Golden Cross (50/200 MA)
Final value: $251898.38
Total invested: $120000.00
Total return: 109.92%
CAGR: 3.78%
Max drawdown: 16.05%
Total trades: 17

Strategy: 6-Month Momentum (5%)
Final value: $233042.46
Total invested: $120000.00
Total return: 94.20%
CAGR: 3.37%
Max drawdown: 19.21%
Total trades: 120
```

Figure 13. Head-to-head strategy comparison for SPX.

## AMZN Strategy Results

| Strategy     | Final Value | Total Invested | Total Return | CAGR   | Max Drawdown | Trade s |
|--------------|-------------|----------------|--------------|--------|--------------|---------|
| Fixed SIP    | \$4.03M     | \$120.0K       | 3255.36%     | 19.20% | 63.62%       | 240     |
| Dynamic SIP  | \$6.70M     | \$160.0K       | 4089.10%     | 20.53% | 63.53%       | 240     |
| Golden Cross | \$902.9K    | \$120.0K       | 652.40%      | 10.62% | 46.69%       | 20      |
| Momentum     | \$1.29M     | \$120.0K       | 979.13%      | 12.63% | 40.15%       | 153     |

```

Enter choice: 13
Enter ticker (SPX, AMZN, NVDA): AMZN
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

===== Strategy Comparison for AMZN =====
Strategy: Fixed SIP
Final value: $4026430.27
Total invested: $120000.00
Total return: 3255.36%
CAGR: 19.20%
Max drawdown: 63.62%
Total trades: 240

Strategy: Dynamic SIP (Dip 10%, Rally 20%, x2)
Final value: $6702557.10
Total invested: $160000.00
Total return: 4089.10%
CAGR: 20.53%
Max drawdown: 63.53%
Total trades: 240

Strategy: Golden Cross (50/200 MA)
Final value: $902881.16
Total invested: $120000.00
Total return: 652.40%
CAGR: 10.62%
Max drawdown: 46.69%
Total trades: 20

Strategy: 6-Month Momentum (5%)
Final value: $1294960.46
Total invested: $120000.00
Total return: 979.13%
CAGR: 12.63%
Max drawdown: 40.15%
Total trades: 153

```

Figure 14. Head-to-head strategy comparison for AMZN.

## NVDA Strategy Results

| Strategy     | Final Value | Total Invested | Total Return | CAGR   | Max Drawdown | Trades |
|--------------|-------------|----------------|--------------|--------|--------------|--------|
| Fixed SIP    | \$2.17M     | \$120.0K       | 1709.55%     | 15.58% | 84.60%       | 240    |
| Dynamic SIP  | \$3.54M     | \$168.5K       | 1999.10%     | 16.44% | 83.91%       | 240    |
| Golden Cross | \$2.04M     | \$120.0K       | 1596.86%     | 15.21% | 52.79%       | 21     |
| Momentum     | \$1.62M     | \$120.0K       | 1250.49%     | 13.90% | 49.84%       | 163    |

```

Enter choice: 13
Enter ticker (SPX, AMZN, NVDA): NVDA
Enter monthly capital: 500
Enter start year: 2000
Enter end year: 2020

===== Strategy Comparison for NVDA =====
Strategy: Fixed SIP
Final value: $2171461.98
Total invested: $120000.00
Total return: 1709.55%
CAGR: 15.58%
Max drawdown: 84.60%
Total trades: 240

Strategy: Dynamic SIP (Dip 10%, Rally 20%, x2)
Final value: $3536978.48
Total invested: $168500.00
Total return: 1999.10%
CAGR: 16.44%
Max drawdown: 83.91%
Total trades: 240

Strategy: Golden Cross (50/200 MA)
Final value: $2036226.99
Total invested: $120000.00
Total return: 1596.86%
CAGR: 15.21%
Max drawdown: 52.79%
Total trades: 21

Strategy: 6-Month Momentum (5%)
Final value: $1620584.60
Total invested: $120000.00
Total return: 1250.49%
CAGR: 13.90%
Max drawdown: 49.84%
Total trades: 163

```

Figure 15. Head-to-head strategy comparison for NVDA.

# Discussion of Results

The results show that asset selection had a large effect on final performance. AMZN had the highest final value and CAGR among the tested assets, followed by NVDA and then SPX. This is expected because AMZN and NVDA had large long-term growth during the 2000 to 2019 period.

For SPX, Dynamic SIP produced a higher final value than Fixed SIP, but it also required more total capital. Golden Cross had the lowest drawdown on SPX, showing that trend-following methods can reduce risk, although they may also reduce total return. Momentum also reduced SPX drawdown compared to Fixed SIP, but its final return was lower.

For AMZN and NVDA, buy-and-hold style monthly investing performed extremely well because both assets had strong long-term growth. However, the maximum drawdown values show that these stocks were still much more volatile than SPX.

## Testing and Verification

The program was tested through the command-line menu. Data loading, historical display, BST insertion and traversal, range search, portfolio buy/undo, individual strategy simulations, and strategy comparisons were all tested manually.

| Test Area            | Result                                                         |
|----------------------|----------------------------------------------------------------|
| CSV loading          | SPX, AMZN, and NVDA each loaded 5031 trading days.             |
| Historical display   | Date range display produced correct rows.                      |
| BST insertion        | AMZN and NVDA inserted correctly by annual return.             |
| BST traversal        | Inorder traversal displayed sorted results.                    |
| BST range search     | Range query returned stocks within the requested return range. |
| Portfolio buy        | Cash balance and holdings updated correctly.                   |
| Undo trade           | Undo restored cash and removed the holding.                    |
| Strategy simulations | All four strategies produced valid output.                     |
| Strategy comparison  | Head-to-head comparison worked for SPX, AMZN, and NVDA.        |

## Challenges and Fixes

## Large Volume Parsing

A major issue occurred when SPX only loaded 1540 trading days instead of the expected 5031. The cause was the volume column. Some SPX volume values were too large for long on the Windows compiler, causing parsing exceptions and skipped rows. The fix was to change volume storage to long long and use stoll().

```
long long volume = stoll(fields[volumeIndex]);
```

## Date Range Filtering

The project uses string comparisons for date filtering. This works because the dates are stored in YYYY-MM-DD format, where lexicographic order matches chronological order.

```
return date >= startDate && date <= endDate;
```

## Strategy Consistency

The strategy comparison menu was updated so that it could compare strategies for any loaded ticker, not only SPX. This required checking both the ETF manager and the Stock manager before passing the correct PriceHistory pointer into the strategies.

## Work Split

The project was completed as a team effort. Biheng Chen completed a large portion of the base project structure, including many of the supporting classes and data management pieces. Irfan Chowdhury focused on strategy implementation, integration, debugging, testing, report preparation, and final verification. Both parts were integrated and tested together through the final menu-driven program.

## AI Usage Statement

AI assistance was heavily used as a support tool during the project. It helped with planning, debugging, explaining errors, organizing code changes, checking strategy logic, and drafting documentation. A lot of the 'unknown' aspects were tackled using AI. The code was still manually integrated, tested, corrected, and verified through the compiler and program outputs. The final results and screenshots were produced by running the completed C++ program.

## Conclusion

The Stock Market Portfolio Simulator successfully demonstrates a complete C++ application built around core ESE 224 concepts. The final program loads historical market data, stores it using linked lists, organizes stock returns through a BST, manages a portfolio, supports undo operations with a stack, and compares multiple trading strategies.

The final version was tested with SPX, AMZN, and NVDA data over the 2000 to 2020 project window. The results show how different strategies can affect return and drawdown depending on the asset. Overall, the project provided practical experience with large program design, data structures, debugging, financial simulation, and collaborative software development.

## Appendix A: Compile and Run Instructions

The project can be compiled from the project root using MSYS2 MINGW64 or a compatible C++ compiler.



```
g++ -std=c++17 -linclude src/*.cpp main.cpp -o stocksim.exe
```

Run the program with:

```
./stocksim.exe
```

## Appendix B: Important Menu Inputs Used for Testing

Load all data

```
1  
4
```

Run Fixed SIP on SPX

```
12  
SPX  
500  
2000  
2020  
1
```

Compare all strategies on AMZN

```
13  
AMZN  
500  
2000  
2020
```

Compare all strategies on NVDA

```
13  
NVDA  
500
```

2000  
2020